8 Automatic Annotations of Data

Goal. Our raw corpus consists of long, continuous Indian urban walking videos (20–30 minutes each), which we partition into short clips and then automatically attach lightweight semantic metadata. These annotations are not treated as ground-truth labels; rather, they serve three purposes: (i) indexing and search over millions of seconds of video, (ii) stratified sampling (e.g., balancing day/night, market/junction), and (iii) label-free evaluation support (e.g., checking whether nearest-neighbor retrievals share similar tags).

Inputs and outputs. Input videos are converted into fixed-length clips (typically $\sim$10 seconds). For each clip, we output a structured tag record containing scene type, time-of-day, weather, crowd/traffic density, road layout, and notable objects, along with per-field confidences and provenance (model version, prompt version).

8.1 Clip generation via shot-aware segmentation

Step 1: decode normalization. We decode each video with a fixed frame rate and resolution (e.g., uniform FPS, fixed short-side), ensuring consistent model inputs and stable downstream statistics. We store decoding metadata (FPS, resolution, codec hash) for reproducibility.

Step 2: scene/shot detection. Continuous walking footage contains turns, occlusions, and occasional hard cuts (intros, overlays). To avoid clips that straddle abrupt transitions, we run shot detection using **:contentReference[oaicite:0]index=0** and only sample clip windows fully contained within a shot boundary.

Step 3: fixed-length clip extraction. Within each shot, we generate 10-second clips (optionally with stride s seconds). Each clip record stores `video_id`, `shot_id`, `t_start`, `t_end`, and a `clip_id`. We also export 1–3 keyframes per clip for qualitative inspection and debugging.

8.2 Structured semantic tagging with a video VLM

Step 4: tagger model. We use a video-capable vision–language model, to attach a compact set of structured tags to each clip. We avoid free-form captions and instead request closed-vocabulary fields to maximize consistency across clips and across time.

Step 5: controlled ontology. Each field is selected from a fixed ontology (with `unsure` allowed). The primary fields are: `scene_type` (market/junction/residential lane/promenade/transit/temple-tourist/highway/alley), `time_of_day` (morning/afternoon/evening/night), `weather` (clear/overcast/rain/fog-haze/unsure), `crowd_density` (low/med/high), `traffic_density` (low/med/high), `road_layout` (intersection/narrow lane/wide road/sidewalk/median/unsure), and `notable_objects` (multi-select list: bus/auto-rickshaw/bike/vendor/police/signage/animals/etc.).

Step 6: prompt format (strict JSON). We require the model to return *only* valid JSON conforming to a schema, forbidding narrative text. A representative prompt (abridged) is: ``Return ONLY valid JSON. Choose values only from the provided lists. If uncertain, set 'unsure'. Provide per-field confidence in [0,1].'' We version prompts to ensure traceability.

8.3 Quality control and calibration

Step 7: self-consistency checks. Automatic tags can be noisy and brittle under camera motion and occlusion. We therefore run a lightweight consistency protocol: (i) duplicate inference with two paraphrased prompts that share the same ontology, and (ii) accept a field only if the two runs agree or if one run exceeds a high-confidence threshold. Otherwise the field is set to `unsure`.

Step 8: confidence thresholding. We apply per-field thresholds (e.g., stricter for `scene_type`, looser for `notable_objects`). This yields a high-precision subset for analysis and a broader, partially-labeled subset for exploratory browsing.

Step 9: human spot-checking. To estimate noise rates, we sample a small audit set stratified by predicted `scene_type` and `time_of_day`. We report approximate precision and the most common failure modes (e.g., confusing commercial streets with markets; night glare inflating "traffic density").

Table 11: **Automatic tag schema per 10s clip.** Tags are weak annotations used for indexing and stratified evaluation; they are not treated as ground truth.

Field	Type / examples
`clip_id`	unique identifier
`scene_type`	market, junction, residential_lane, promenade, transit, temple_tourist, highway, alley, unsure
`time_of_day`	morning, afternoon, evening, night, unsure
`weather`	clear, overcast, rain, fog_haze, unsure
`crowd_density`	low, medium, high, unsure
`traffic_density`	low, medium, high, unsure
`road_layout`	intersection, narrow_lane, wide_road, sidewalk_present, median_present, unsure
`notable_objects`	list: bus, auto_rickshaw, bike, vendor, police, signage, animals, ...
`confidence.*`	per-field confidence in $[0, 1]$
`provenance`	model version, prompt version, timestamp

9 Evaluating V-JEPA on Indian Urban Walking Clips

Setup. We evaluate a frozen video encoder on our Indian urban clip corpus using (i) *label-free* representation tests and (ii) *tag-conditioned* tests using weak, structured annotations produced by . Tags are treated as *noisy metadata* (not ground truth), so we report results on a high-confidence subset and include sensitivity to confidence thresholds.

9.1 Step-by-step evaluation protocol

Step 1: clip bank and splits (avoid leakage). We split each long video (sourced from, e.g., the collection) into $\sim$10s clips using shot-aware segmentation. We define train/val/test splits *by video id* (not by clip) to prevent near-duplicate temporal windows from appearing in both query and retrieval pool. For retrieval evaluation, we additionally define an exclusion window Δt (e.g., ± 30s) within the same video to avoid trivial adjacency matches.

Step 2: embedding extraction (frozen encoder). For every clip i, we extract a fixed-dimensional embedding $e_i \in \mathbb{R}^d$ from the frozen encoder by pooling spatiotemporal patch features. We L2-normalize embeddings so cosine similarity is the primary distance measure.

Step 3: build the retrieval index. We store all embeddings in a kNN index. For each query clip q, we retrieve top-K nearest neighbors subject to the chosen leakage constraints (easy vs hard mode; see below).

Step 4: define evaluation subsets using automatic tags. Each clip has structured tags (scene type, time-of-day, weather, crowd/traffic density, road layout, notable objects) with per-field confidence. We define: (i) a *high-confidence subset* (e.g., `conf(scene_type)` $\geq \tau$), (ii) stratified slices (e.g., night-only, high-crowd, market-only), and (iii) an *unknown/unsure* bucket that is excluded from class-wise scores but included for overall unlabeled tests.

9.2 Overall (label-free) evaluation

Step 5: qualitative kNN sanity grids. We sample $\sim$20 queries across diverse clips and visualize the query keyframe plus top-K neighbor keyframes. We inspect whether neighbors are coherent along interpretable axes (street geometry, lighting, crowd/traffic regime), and we explicitly repeat the visualization in hard mode (excluding same-video near-time neighbors).

Step 6: cycle consistency (neighborhood coherence). For a clip A, let $B = \text{NN}(A)$ and $C = \text{NN}(B)$. We define cycle success if $C = A$ (or $C \in \text{Top}k(A)$). We report:

$$\text{Cycle@}k = \Pr\left[A \in \text{Top}k(\text{NN}(A))\right]$$

in two modes: *Easy:* allow same-video neighbors; *Hard:* disallow same-video neighbors within Δt.

Step 7: neighborhood stability under augmentations (robustness). For each clip A, we create two benign views $A^{(1)}, A^{(2)}$ (crop/resize) and compute:

$$\text{Overlap@}K(A) = \frac{\left|\text{TopK}(A^{(1)}) \cap \text{TopK}(A^{(2)})\right|}{K}, \qquad \text{Overlap@}K = \mathbb{E}_A[\text{Overlap@}K(A)].$$

High overlap indicates invariance to nuisance transformations.

Step 8: embedding-space clustering diagnostics (optional). We run k-means (or hierarchical clustering) on embeddings and report unsupervised cohesion/separation metrics (e.g., silhouette score). This is label-free and complements retrieval; it is especially useful to detect collapse modes (e.g., clusters dominated by illumination only).

9.3 Class-wise evaluation using weak tags

Step 9: tag-agreement retrieval scores (per class). Using `scene_type` as the primary class label on the high-confidence subset, we measure whether neighbors share the same tag as the query. For a query clip q with class $c(q)$ and retrieved neighbors $\{n_j\}_{j=1}^{K}$, define:

$$\text{Prec@}K(q) = \frac{1}{K}\sum_{j=1}^{K} \mathbb{I}\left[c(n_j) = c(q)\right], \qquad \text{Prec@}K = \mathbb{E}_q[\text{Prec@}K(q)].$$

Class-wise: we also report Prec@K *conditioned on class*:

$$\text{Prec@}K(c) = \mathbb{E}_{q:\, c(q)=c}[\text{Prec@}K(q)].$$

This directly answers: *for markets (or junctions), do we retrieve other markets?*

Step 10: mAP@K / nDCG@K with tag relevance (ranking quality). To evaluate ranking quality beyond top-1, we treat a neighbor as *relevant* if it matches the query tag (or matches a subset of tags, e.g., scene type + time-of-day). We then compute: - *mAP@K:* mean average precision over queries - *nDCG@K:* if we assign graded relevance (e.g., +2 if scene_type matches, +1 if time_of_day matches)

These are robust to the fact that multiple clips can be relevant.

Step 11: multi-attribute slice metrics (beyond scene type). We repeat Step 9–10 for other fields (time-of-day, weather, crowd density, traffic density, road layout), always on high-confidence slices. This yields a matrix of results that reveals what the representation captures best (e.g., strong time-of-day, weaker road layout).

Step 12: confusion-style analysis via retrieval neighborhoods. For each class c, we collect the distribution of neighbor tags among top-K results (in hard mode) and report the dominant confusions (e.g., `market_street` often retrieving `commercial_high_street`). This is more informative than a single scalar score when tags are noisy.

9.4 Reporting: overall vs class-wise

Step 13: overall aggregation. We report: - overall metrics: Prec@K, mAP@K, nDCG@K, Cycle@k, Overlap@K - and the *macro average* across classes (treat classes equally), plus *micro average* (weighted by class frequency)

Step 14: robustness checks. We include: - *Confidence threshold sweep:* evaluate for $\tau \in \{0.5, 0.6, 0.7, 0.8\}$ to show stability to tagging noise. - *Hard vs easy retrieval:* report both, since easy mode can be inflated by temporal adjacency.

Step 15: minimal student-friendly protocol. Students can implement the evaluation with three deliverables: (i) neighbor grids (qualitative), (ii) Cycle@$\{1, 10\}$ and Overlap@20 (label-free), and (iii) class-wise Prec@20 for `scene_type` on the high-confidence subset (tag-conditioned).

10 Continual Self-Supervised Pretraining on Indian Urban Clips

Goal. We continue pretraining a pretrained V-JEPA model on our Indian urban walking corpus using the *same* latent-prediction objective as the original method. The purpose is to adapt the representation to domain-specific statistics—handheld motion, dense crowds, mixed traffic, glare/night lighting, heavy occlusion, signage, and long-tailed street objects—while preserving general-purpose semantic invariances. This section specifies an implementable recipe: how clips are formed, how context/target views and masks are sampled, how the student–teacher loss is computed, and which weights constitute the final "V-JEPA on Indian data" checkpoint.

10.1 Training data and sampling

Clip bank. Long-form videos are partitioned into shot-consistent 10s clips, yielding an unlabeled dataset $\mathcal{D} = \{x_i\}_{i=1}^{N}$. Each clip stores provenance (`video_id`, start/end timestamps, optional `shot_id`) and 1–3 keyframes for auditing. We split train/validation strictly by `video_id` so that clips from the same source video never appear in both sets, avoiding near-duplicate leakage due to temporal adjacency.

Decode normalization. To keep training stable, all clips are decoded with a consistent policy. We fix (i) a decode FPS, (ii) a fixed number of sampled frames per clip T, and (iii) a fixed spatial preprocessing pipeline (resize + crop). Each training sample is therefore a tensor of shape $T \times H \times W \times 3$ with consistent T, H, W across the dataset.

Sampling policy. By default, we sample clips uniformly over `video_id` (and then uniformly over clips within each video) to prevent a few long videos from dominating the update distribution. If structured VLM tags are available, we use them only for *stratified batching* (e.g., maintaining a stable mix across day/night, market/junction, low/high crowd) so continual pretraining does not drift toward the most frequent conditions. These tags are treated as metadata and are not used as supervised targets.

10.2 Model components and what is trained

Student encoder (trainable). The pretrained V-JEPA encoder f_θ is the **student**. It is updated by backpropagation and is the model that learns domain adaptation on Indian clips.

Teacher encoder (training-time target, not trainable). We maintain a second encoder $f_{\bar{\theta}}$ with identical architecture as a *teacher* that provides stable regression targets. The teacher is not updated by gradients; it is updated only by exponential moving average of the student parameters.

Predictor (trainable). A small predictor g_ϕ maps student features into the teacher feature space. This is trained jointly with the student.

Which checkpoint is the final "V-JEPA on Indian data." The **resultant Indian-adapted V-JEPA model** is defined to be the **student encoder** f_θ at the selected checkpoint, because the student is the only network optimized on our data. The teacher is an auxiliary training stabilizer and is not used as the final model unless explicitly stated otherwise.

10.3 Continual JEPA objective (implementable)

Two views per clip. For each clip x, we generate two spatiotemporal views: a *context view* $v_c \sim \mathcal{T}_c(x)$ and a *target view* $v_t \sim \mathcal{T}_t(x)$. The transforms $\mathcal{T}_c, \mathcal{T}_t$ are video-consistent augmentations applied with the same parameters across frames (e.g., one crop for all frames), so temporal structure is preserved.

Spatiotemporal masking. The encoder operates on a grid of spatiotemporal tokens (tubelets/patches). We sample: (i) a *target mask* M_t specifying which target tokens must be predicted, and (ii) a *context mask* M_c specifying which tokens are visible to the student. A practical choice is to sample M_t as a union of a small number of rectangular spatiotemporal blocks on the token grid (masking 15%–30% of tokens), then set M_c to the complement (optionally dropping an additional small fraction of visible tokens to increase difficulty). This creates a prediction problem: infer the latent representation of masked regions from the remaining context.

Student–teacher latent regression. Let $S = f_\theta(v_c; M_c)$ denote student features computed from the context view, and let $T = f_{\bar{\theta}}(v_t; M_t)$ denote teacher features on masked target tokens (computed

without gradients). The predictor produces $\hat{T} = g_\phi(S)$ aligned to the same masked token set. We minimize mean-squared error on the masked targets:

$$\mathcal{L}_{\text{JEPA}}(\theta, \phi) = \mathbb{E}\Big[\|\hat{T} - \text{sg}(T)\|_2^2\Big],$$

averaging over masked tokens and then over the minibatch. The stop-gradient makes the teacher a stable target and prevents collapse in non-contrastive latent regression.

Teacher update by EMA. After each student optimizer step, the teacher is updated by EMA:

$$\bar{\theta} \leftarrow \tau\bar{\theta} + (1-\tau)\theta, \qquad \tau \in [0.99, 0.9999].$$

This ensures the target encoder evolves smoothly as the student adapts to the Indian domain.

10.4 Optimization details (runnable defaults)

Optimizer and learning rates. We optimize student parameters θ and predictor parameters ϕ with a standard optimizer such as AdamW. We use an adaptation-scale learning rate for the backbone (small compared to training from scratch) and a slightly larger rate for the predictor. We apply short warmup at the start of training and gradient clipping to limit occasional spikes caused by abrupt illumination changes and heavy occlusion.

Conservative drift control (optional). To prevent over-specialization, we optionally anchor the student to its pretrained initialization $\theta^{(0)}$:

$$\mathcal{R}_{\text{stab}}(\theta) = \lambda\|\theta - \theta^{(0)}\|_2^2, \qquad \min_{\theta,\phi} \mathcal{L}_{\text{JEPA}}(\theta, \phi) + \mathcal{R}_{\text{stab}}(\theta).$$

This stabilizer biases learning toward the smallest parameter change that explains Indian-domain statistics; we tune λ in ablations to trade off domain gain versus retention.

Concrete implementable defaults. A practical starting configuration is: clip length 10s; $T \in \{16, 32\}$ frames; input resolution 224 (or 256); target mask ratio 0.20 with 2–6 spatiotemporal blocks; mixed precision; EMA momentum τ warmed from $\sim$0.996 to $\sim$0.999; gradient clip 1.0; checkpoint every 2k–5k steps. These choices are directly implementable in standard training code (DDP + AMP).

10.5 Training loop (step-by-step)

One training step. Each step implements the following sequence:

- Sample a minibatch of clips from $\mathcal{D}$ (uniform over `video_id`).
- Decode/sample T frames per clip and apply the spatial resize/crop pipeline.
- Generate $v_c = \mathcal{T}_c(x)$ and $v_t = \mathcal{T}_t(x)$ using video-consistent transforms.
- Sample masks (M_c, M_t) on the token grid.
- Student forward: $S = f_\theta(v_c; M_c)$.
- Teacher forward (no grad): $T = f_{\bar{\theta}}(v_t; M_t)$.
- Predictor: $\hat{T} = g_\phi(S)$; compute $\mathcal{L}_{\text{JEPA}}$ (plus $\mathcal{R}_{\text{stab}}$ if used).
- Backprop + optimizer step on (θ, ϕ).
- EMA update of teacher parameters $\bar{\theta}$.

Checkpointing. We save both student and teacher weights at each checkpoint for reproducibility; however, the **student** checkpoint is the official "V-JEPA-Indian" model used downstream.

10.6 Validation and model selection

Fast validation subset. We maintain a fixed validation subset of clips sampled by `video_id` (e.g., 5–10k clips) so retrieval metrics are cheap and comparable across checkpoints.

Selection criteria. We select checkpoints based on retrieval quality and robustness on unaltered validation clips: Cycle@$\{1, 10\}$ under hard-mode (excluding same-video near-time neighbors) and neighborhood stability under augmentations (Overlap@K). If weak tags exist, we additionally report slice-wise trends (scene type, time-of-day) as diagnostics, but do not treat them as supervised accuracy targets.

10.7 Reporting and ablations

Primary metrics. We report label-free metrics (Cycle@$\{1, 10\}$, Overlap@K) and tag-conditioned neighborhood agreement on high-confidence slices to summarize how continual pretraining reshapes retrieval neighborhoods for Indian streets.

Ablations. We ablate: (i) total training steps, (ii) augmentation strength in $\mathcal{T}_c, \mathcal{T}_t$, (iii) EMA momentum τ, and (iv) stabilizer weight λ. For each, we report overall retrieval metrics and the most informative slices (scene type and time-of-day) to characterize where adaptation helps most and where it risks over-specialization.

11 Surgery Fine-Tuning: Progressive Prefix Unfreezing with Factor Datasets

Goal. We fine-tune V-JEPA on Indian urban walking videos using the *same* self-supervised latent-prediction loss as continual pretraining, but with two additional controls that make the procedure "surgical": (i) **progressive prefix unfreezing** (only layers $0 \dots n$ are trainable; deeper layers are frozen), and (ii) **selective factor patching** (each training sample is rendered to emphasize either **layout**, **agents**, or **interactions**). The objective remains unchanged; only the *input distribution* and the *trainable depth* are controlled. The final Indian-adapted model used downstream is always the **student encoder** checkpoint (the only network updated by gradients).

11.1 Factor datasets from SAM segmentation

Inputs. Each raw 10s clip x is decoded into T frames with fixed preprocessing (FPS, resize, crop) matching Sec. 10. We assume $T \in \{16, 32\}$.

Segmentation. For each frame t, run SAM3 to obtain instance masks $\{m_{t,k}\}_{k=1}^{K_t}$ with optional confidence scores. We store masks as RLE/PNG for reproducibility.

Tracking (minimal, implementable). We associate masks across frames into tracklets using greedy IoU matching: for each mask in frame $t{+}1$, match to the tracklet whose last mask has maximum IoU, subject to IoU $\geq \delta_{\text{iou}}$ (default 0.3). Allow short gaps of up to g frames (default $g{=}1$) by keeping unmatched tracklets alive briefly. This yields tracklets $\{\tau_j\}$; each tracklet has a per-frame mask when visible.

Agent vs layout separation (practical rule). Since SAM is class-agnostic, we determine *agents* using a motion-based filter on each tracklet: compute centroid displacement (or mean optical flow magnitude) inside the mask across consecutive frames and define a motion score. A tracklet is declared an *agent* if its motion score exceeds a threshold for at least r_m frames (defaults: $r_m{=}4$ frames and motion threshold set to a small fraction of frame size). All remaining pixels are treated as layout/background.

Per-frame masks. Let A_t be the union of agent masks in frame t, and B_t its complement (layout/background). We optionally dilate A_t by a few pixels to preserve thin structures (legs, bike spokes) and to avoid mask under-coverage.

Derived datasets. From each raw clip we generate three factor datasets (stored as either cached videos or deterministic re-render recipes):

- **Layout-only** $\mathcal{D}_L$**:** suppress A_t and preserve B_t.
- **Agent-only** $\mathcal{D}_A$**:** preserve A_t and suppress B_t.
- **Interaction-focused** $\mathcal{D}_I$**:** mine multi-agent interaction events (below) and extract interaction tubes/crops.

Each derived sample stores provenance: `video_id`, `clip_id`, timestamps, and if applicable `tracklet_ids` for the interaction pair.

11.2 Mining interaction events for $\mathcal{D}_I$

Why selection instead of heavy editing. Interaction is inherently spatiotemporal; rather than synthesizing interactions, we *select* subclips where interactions naturally occur and then crop them to focus the model on relational motion.

Candidate pairs. For each clip, consider pairs of agent tracklets (τ_a, τ_b) that overlap in time for at least r frames (default $r{=}4$).

Distance and persistence. For each overlapping frame, compute the centroid distance d_t between the two masks (or between their boxes). A pair is a candidate interaction if d_t stays below a threshold $d_{\max}$ for at least r consecutive frames. A practical default is to set $d_{\max}$ relative to the frame width (e.g., 0.15–0.25 of width), so it scales with resolution.

Relative motion cue (prevents accidental near passes). Require at least one simple relational cue over the persistence window:

- **Approach/retreat:** d_t decreases then increases (or monotonic decrease) across the window.
- **Crossing:** the direction vectors of the two centroids have a large angle (e.g., $> 45^\circ$).
- **Following:** velocity directions are similar and one centroid remains behind the other along the motion direction.

This is deliberately lightweight: it is enough to avoid selecting "two agents happen to be close in one frame".

Interaction tube. For a confirmed pair, define a per-frame box that encloses both masks, then expand by a margin m (default 10–20%). The union of these boxes over the event window defines a spatiotemporal tube. Extract the tube as a cropped clip; optionally resize back to the model input size. Store the tube boxes and time window as metadata.

11.3 Selective factor patching per training sample

Core rule. Each training sample chooses exactly one factor mode $m \in \{L, A, I\}$. We apply a patch operator $\mathcal{P}_m$ to the raw clip x to obtain a patched clip $x^{(m)} = \mathcal{P}_m(x)$, then run the *standard V-JEPA training step* on $x^{(m)}$. Patching is therefore an *input transformation*, not supervision and not a new loss.

Layout patch ($m{=}L$). For each frame t, suppress pixels inside A_t using either: (i) strong blur/mosaic (default, cheapest, avoids inpainting artifacts), or (ii) inpainting (optional, higher quality, more compute). Everything outside A_t is unchanged.

Agent patch ($m{=}A$). For each frame t, preserve pixels inside A_t and suppress background B_t using either: (i) hard replacement (zeros / mean color), or (ii) soft matte attenuation (multiply background by a small factor). Soft matte reduces boundary artifacts and makes training less sensitive to segmentation imperfections.

Interaction patch ($m{=}I$). Sample an interaction tube instance from $\mathcal{D}_I$ and extract the crop. Use one of two renderings: (i) **raw tube crop** (keeps local context like lane markings), or (ii) **masked tube crop** (soft matte suppresses background inside the crop so agents dominate locally). We mix these renderings so the model cannot rely on a single visual signature of "interaction" samples.

Interaction patch perturbations. To prevent shortcut learning from deterministic crops or mask boundaries, we apply light perturbations to $x^{(I)}$ (randomly choose 1–3 per sample):

- **Tube jitter:** random spatial offset and margin jitter (e.g., $\pm$5–15% of box size); optional temporal jitter by $\pm$1–2 frames.
- **Context margin randomization:** vary crop margin so the interaction sometimes includes more road context, sometimes less.
- **Raw vs masked mixing:** choose raw crop vs soft-matted crop with fixed probabilities (e.g., 0.5/0.5).

- **Soft boundary blending:** feather mask edges when suppressing background to avoid crisp contours as shortcuts.
- **Mask noise:** small random dilation/erosion to break over-reliance on exact contours.
- **Artifact realism:** mild motion blur/compression noise inside the crop with small probability to match real walking-video artifacts.

These perturbations preserve the interaction event while destroying superficial regularities of the patching pipeline.

11.4 Training objective (unchanged V-JEPA loss)

Student, teacher, predictor. The student encoder f_θ is the model being optimized. The teacher encoder $f_{\bar{\theta}}$ is an EMA copy used only for stable regression targets. The predictor g_ϕ maps student features to teacher feature space and is trained jointly with the student.

Views and masks (run on the patched clip). Given a patched clip $x^{(m)}$, we sample: (i) a context view v_c and (ii) a target view v_t using video-consistent augmentations (one crop per clip, applied to all frames), plus lightweight color/blur transforms. We then sample spatiotemporal masks on the token grid: a target mask M_t (tokens to predict) and a context mask M_c (tokens visible to the student). A default is 15–30% target tokens sampled as 2–6 spatiotemporal blocks; M_c is the complement (optionally dropping a small extra fraction to increase difficulty).

Latent regression step. Compute student features on context, teacher features on masked targets (no gradient), predict teacher targets from student features using g_ϕ, and minimize mean squared error over masked target tokens. After the optimizer step, update teacher weights by EMA with momentum τ.

Which checkpoint is the final model. The final Indian-adapted model is the **student encoder** f_θ checkpoint selected by validation retrieval. The teacher is a training-time stabilizer and is not used as the official downstream model unless explicitly stated otherwise.

11.5 The surgery: progressive prefix unfreezing

What is trainable. Let the backbone have L transformer blocks. At stage s, we choose a prefix boundary n_s: only layers $\ell \leq n_s$ are trainable; layers $\ell > n_s$ are frozen (no gradients, excluded from optimizer updates).

How to implement freezing correctly. In practice: (i) set `requires_grad=False` for parameters in layers $\ell > n_s$, (ii) exclude them from optimizer parameter groups (or set LR to zero and disable weight decay for safety), (iii) ensure frozen-layer optimizer state is not updated. This prevents accidental drift in frozen blocks.

Stage schedule aligned to factors. We run three stages with increasing trainable depth and factor dominance:

- **Stage 1 (Layout):** n_1 shallow; mode mixture dominated by L.
- **Stage 2 (Agent):** $n_2 > n_1$; mixture dominated by A with replay of L.
- **Stage 3 (Interaction):** $n_3 > n_2$; mixture dominated by I with replay of A and L.

A simple starting point is $n_1 \approx 0.25L, n_2 \approx 0.50L, n_3 \approx 0.75L$.

Mode mixture (explicit defaults). Each stage samples factor modes with fixed probabilities: Stage 1: $p(L) \approx 1.0$; Stage 2: $p(A) \approx 0.9, p(L) \approx 0.1$; Stage 3: $p(I) \approx 0.85, p(A) \approx 0.10, p(L) \approx 0.05$. This is implemented as: sample m first, then sample a clip from the corresponding dataset and apply $\mathcal{P}_m$.

Layer-wise learning rate decay (recommended). Within the unfrozen prefix, apply a smaller LR to earlier layers and a larger LR near the boundary. This reduces the chance of destroying generic low-level filters while still allowing meaningful adaptation where capacity is most needed.

11.6 Stage-wise training loop (runnable, step-by-step)

Per-stage initialization. At the start of each stage: (i) increase the trainable prefix boundary to n_s, (ii) rebuild optimizer param groups so newly-unfrozen layers are included, (iii) run a short warmup (a few hundred steps) to avoid loss spikes from newly-trainable layers.

One training iteration. For each iteration:

- Sample factor mode m according to the current stage mixture.
- Sample a raw clip x (or an interaction tube instance when $m{=}I$).
- Render patched clip $x^{(m)} = \mathcal{P}_m(x)$ (apply interaction perturbations if $m{=}I$).
- Sample context/target views v_c, v_t and masks M_c, M_t from $x^{(m)}$.
- Run student forward on (v_c, M_c); run teacher forward on (v_t, M_t) with no gradient.
- Predict teacher targets using the predictor; compute latent regression loss over masked target tokens.
- Backprop and update only the unfrozen prefix layers and predictor parameters.
- Update teacher weights by EMA.

Checkpointing. Save student and teacher weights regularly. Use the student checkpoint for evaluation and as the final output model.

11.7 Recommended hyperparameter defaults (so others can reproduce)

Clip and masking. 10s clips, $T \in \{16, 32\}$ frames, resolution 224 (or 256), target-mask ratio 0.20, 2–6 spatiotemporal mask blocks, mixed precision, gradient clipping (e.g., 1.0).

Optimization. Use an adaptation-scale backbone LR (small) and a slightly larger predictor LR. Apply short warmup at each stage start. EMA momentum τ in the high range (e.g., 0.99+), optionally warmed up from a slightly lower value early in training to speed teacher tracking.

Interaction mining thresholds. Defaults that are easy to implement and tune: $r{=}4$ consecutive frames for persistence, $d_{\max}$ set as a fixed fraction of frame width, and a simple relative-motion cue (approach/cross/follow).

Quality filters. Drop agent-only samples where $|A_t|$ is nearly zero across most frames; drop layout-only samples where $|A_t|$ covers too much of the image (layout disappears); drop interaction events that do not persist (fail the r-frame rule) or have broken tracking.

11.8 How we verify surgery worked

Overall retrieval. Evaluate the student encoder with the same label-free retrieval suite: qualitative neighbor grids, hard-mode Cycle@$\{1, 10\}$ (exclude same-video near-time neighbors), and augmentation stability Overlap@K.

Factor-sliced retrieval. Query with samples from $\mathcal{D}_L$, $\mathcal{D}_A$, and $\mathcal{D}_I$ separately. A successful surgery schedule yields: (i) layout-consistent neighborhoods for layout queries after Stage 1, (ii) agent-consistent neighborhoods for agent queries after Stage 2, (iii) interaction-consistent neighborhoods for interaction queries after Stage 3, with minimal regression on earlier factors due to replay mixing and conservative depth unfreezing.

Sanity check against patch shortcuts. Repeat retrieval evaluation using (i) raw clips only and (ii) patched clips only. Improvements should transfer to raw clips; if gains exist only on patched clips, the model is likely exploiting patch artifacts rather than learning the intended factor.